13 Kuis Formatif

S1 Pendidikan Matematika — FKIP Universitas Sultan Ageng Tirtayasa

Nama: Fahrezi Sauqi Alghani
NIM: 2225250029
Kelas: 3E

1. **Apa perbedaan syntax error, runtime error, dan logical error?**

   Perbedaan ketiganya terletak pada waktu terjadinya kesalahan dan dampaknya terhadap jalannya program:

Jenis Error	Waktu Terjadi	Penyebab & Dampak
Syntax Error	Sebelum kode dijalankan (saat *compilation/parsing*).	Pelanggaran aturan tata bahasa pemrogramannya (misal: kurang tanda titik dua `:` atau salah ejaan kata kunci). Program **tidak bisa berjalan sama sekali**.
Runtime Error	Saat program sedang berjalan (*execution phase*).	Operasi ilegal yang tidak dapat diproses komputer (misal: pembagian dengan nol `ZeroDivisionError` atau variabel tak terdefinisi). Program **berhenti mendadak (crash)**.
Logical Error	Saat program berjalan sampai selesai.	Kesalahan pada logika algoritma (misal: salah rumus luas). Program berjalan tanpa error/crash, namun **menghasilkan keluaran yang salah**.

2. **Apa fungsi traceback?**

   Fungsi utama `traceback` adalah memberikan Laporan Jejak Eksekusi (*call stack*) ketika terjadi error pada saat program berjalan (runtime error). Traceback menampilkan urutan pemanggilan fungsi, nomor baris kode yang mengalami kesalahan, serta jenis pesan error spesifiknya, sehingga memudahkan programmer dalam melacak dan menemukan lokasi persis terjadinya masalah.

3. **Kapan Step Into digunakan?**

   `Step Into` digunakan saat proses *debugging* ketika eksekusi kode berada di baris pemanggilan suatu fungsi, dan programmer ingin **masuk ke dalam bagian dalam (internal) fungsi tersebut** untuk memeriksa bagaimana alur logika dan variabel di dalam fungsi itu diproses baris demi baris.

4. **Apa beda Step Over dan Step Into?**

   - **Step Over:** Menjalankan fungsi di baris saat ini secara keseluruhan tanpa masuk ke bagian dalam fungsinya, lalu langsung berhenti di baris berikutnya. Digunakan jika kita yakin fungsi tersebut sudah benar.
   - **Step Into:** Masuk ke dalam baris pertama di dalam fungsi yang sedang dipanggil untuk menganalisis jalannya eksekusi fungsi baris demi baris.

5. **Mengapa nilai batas perlu dimasukkan dalam test case?**

   Nilai batas (*boundary value*) perlu dimasukkan karena kesalahan logika (*logical bugs*) paling sering terjadi pada titik-titik batas kondisi (misalnya penggunaan operator `<` vs `<=`). Menguji nilai batas (seperti batas minimum, batas maksimum, dan titik tepat pada batas) memastikan bahwa sistem tetap valid dalam menangani transisi kondisi.

6. Apa beda expected result dan actual result?

- **Expected Result (Hasil yang Diharapkan):** Output atau perilaku sistem yang seharusnya terjadi berdasarkan rancangan/spesifikasi kebutuhan sebelum pengujian dilakukan.
- **Actual Result (Hasil Aktual):** Output atau perilaku sistem yang benar-benar dihasilkan oleh program secara nyata saat pengujian dijalankan. Jika *expected result* ≠ *actual result*, maka terdapat bug.

7. Apa yang dimaksud regresi setelah perbaikan?

Regresi (Regression) setelah perbaikan adalah munculnya masalah/bug baru atau rusaknya fitur lain yang sebelumnya berjalan normal akibat adanya perubahan/perbaikan kode yang baru saja dilakukan. Pengujian regresi (*regression testing*) dilakukan untuk memastikan perbaikan kode tidak merusak bagian program yang lain.

8. Mengapa debugging perlu diawali dengan reproduksi masalah?

Debugging harus diawali dengan mereproduksi masalah agar programmer dapat:

- Memastikan dan mengonfirmasi bahwa bug tersebut benar-benar ada secara konsisten.
- Memahami kondisi/langkah-langkah spesifik dan input apa yang memicu terjadinya error.
- Memverifikasi kelak apakah perbaikan yang diterapkan benar-benar telah menyelesaikan masalah tersebut.

9. Mengapa perubahan kode sebaiknya minimal saat memperbaiki bug?

Perubahan kode sebaiknya diminimalkan (fokus langsung ke akar masalah) untuk:

- Meminimalkan risiko terciptanya regresi atau bug baru di bagian lain.
- Memudahkan proses peninjauan ulang kode (*code review*) dan lacak balik (*tracking/git diff*).
- Memastikan bahwa perubahan tersebut benar-benar menjadi solusi langsung tanpa mengubah perilaku sistem secara tidak sengaja.

10. Apa manfaat commit khusus bertipe fix pada GitHub?

Manfaat membuat commit khusus bertipe `fix` (misal: `fix: resolve zero division in average score calculation`) adalah:

- **Riwayat Jelas (Semantic Commits):** Memudahkan tim pengembang memahami riwayat perubahan bahwa commit tersebut berisi perbaikan bug.
- **Dokumentasi Otomatis:** Memudahkan pembuatan catatan rilis (*changelog*) secara otomatis.
- **Isolasi Perubahan:** Memisahkan perbaikan bug dari penambahan fitur baru (`feat`), sehingga jika terjadi masalah, rollback commit dapat dilakukan dengan aman.